

ALBERI BINARI DI RICERCA

PIETRO DI LENA

DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
UNIVERSITÀ DI BOLOGNA

ALGORITMI E STRUTTURE DI DATI
ANNO ACCADEMICO 2024/2025



- Alberi Binari di Ricerca (**BST**, dall'inglese Binary Search Tree):
 - Alberi binari radicati con vincoli sull'organizzazione delle chiavi
 - Permette una ricerca binaria sulla struttura Albero Binario
- Vedremo come implementare le operazioni basilari di ricerca, inserimento e rimozione su un Albero Binario di Ricerca
- Tali operazioni hanno un costo proporzionale all'altezza dell'albero

ALBERI BINARI DI RICERCA (~1960)

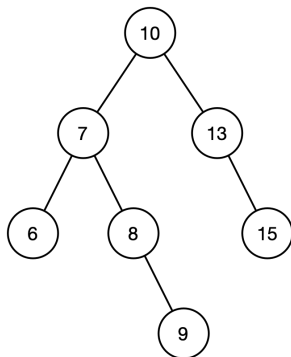
- **Idea:** portare la ricerca binaria su una struttura concatenata

- Definizione di Albero Binario di Ricerca:

- 1 Albero Binario

- 2 Ogni nodo v contiene una **chiave confrontabile** $v.key$ e **dati** $v.data$ associati alla chiave

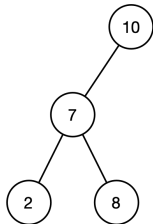
- 3 **Proprietà di ordinamento dei BST:** tutte le chiavi nel sottoalbero sinistro di v sono $\leq v.key$ e tutte le chiavi nel sottoalbero destro di v sono $\geq v.key$



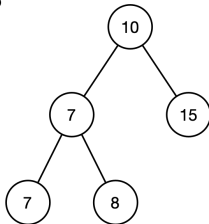
- La proprietà 3 permette di effettuare una ricerca binaria sull'albero
- Quale visita permette di ottenere tutte le chiavi in ordine?

ALBERI BINARI DI RICERCA?

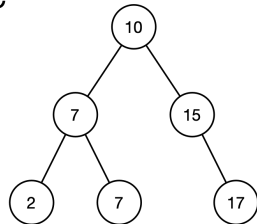
A



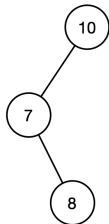
B



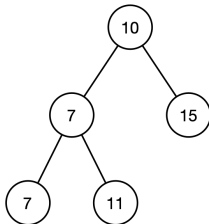
C



D



E



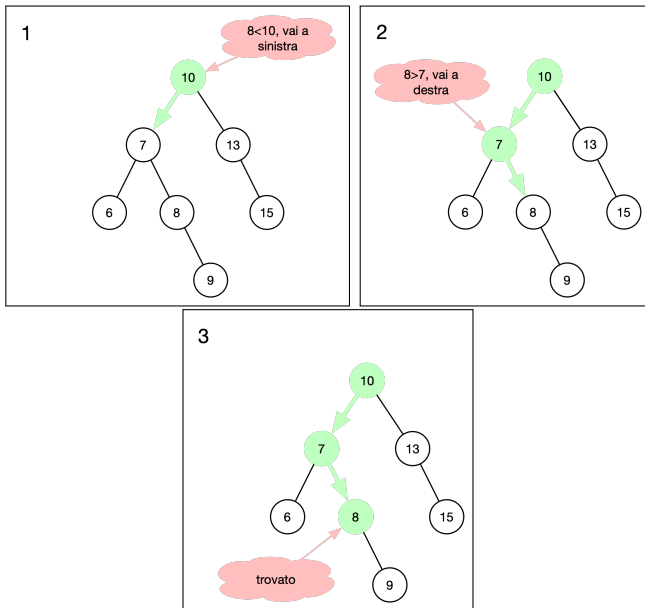
F



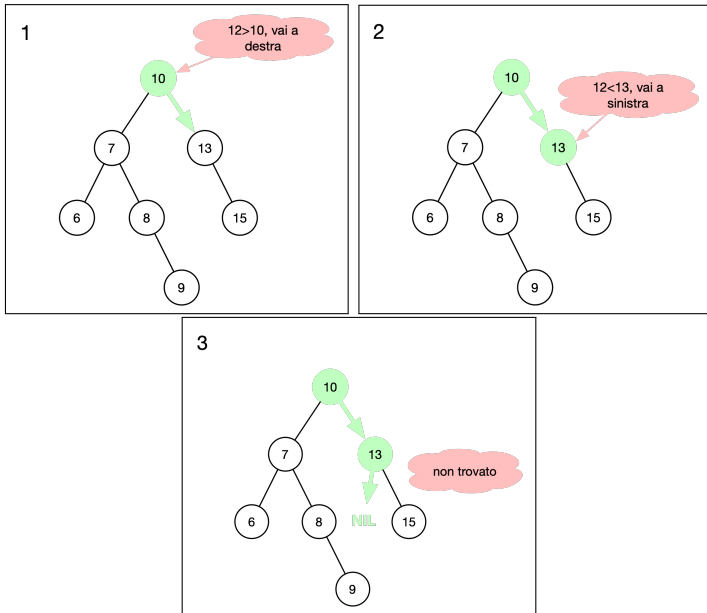
OPERAZIONI SU ALBERI BINARI DI RICERCA

- Operazioni fondamentali su Alberi Binari di Ricerca:
 - $\text{SEARCH}(T, k)$: ritorna il nodo con chiave k in T o NIL
 - $\text{INSERT}(T, k, d)$: inserisce in T (e ritorna) un nodo (k, d)
 - $\text{DELETE}(T, k)$: rimuove (e ritorna) il nodo con chiave k in T
- Operazioni di supporto per la DELETE:
 - $\text{MAX}(T)$: ritorna il nodo con chiave massima k in T
 - $\text{MIN}(T)$: ritorna il nodo con chiave minima k in T
 - $\text{PREDECESSOR}(T)$: ritorna il nodo che precede T quando i nodi sono ordinati rispetto ad una visita in-ordine
 - Se le chiavi sono tutte distinte, è equivalente al nodo avente la più grande chiave $k < T.\text{key}$
 - NIL se $T.\text{key}$ è la chiave minima in T
 - $\text{SUCCESSOR}(T)$: simmetrica a PREDECESSOR

ESEMPIO: RICERCA DEL NUMERO 8



ESEMPIO: RICERCA DEL NUMERO 12



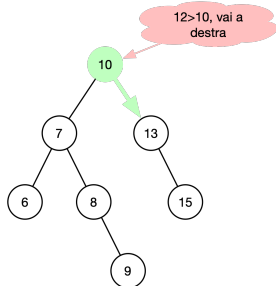
PSEUDOCODICE: SEARCH

```
1: function SEARCH(BST  $T$ , KEY  $k$ )  $\rightarrow$  NODE  
2:    $tmp = T.root$   
3:   while  $tmp \neq \text{NIL}$  do  
4:     if  $k == tmp.key$  then  
5:       return  $tmp$   
6:     else if  $k < tmp.key$  then  
7:        $tmp = tmp.left$   
8:     else  
9:        $tmp = tmp.right$   
10:  return NIL
```

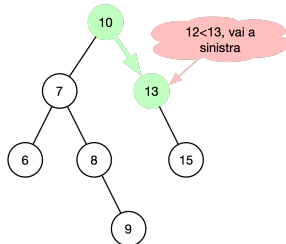
- Ritorna il primo nodo con chiave k
- Costo nel caso ottimo: $O(1)$
 - Quando la radice ha chiave k
- Costo nel caso pessimo: $\Theta(h)$
 - h = altezza dell'albero
 - La visita è sempre confinata su un percorso radice-foglia
 - N.B. $h = O(n)$, n = numero di nodi in T

ESEMPIO: INSERIMENTO

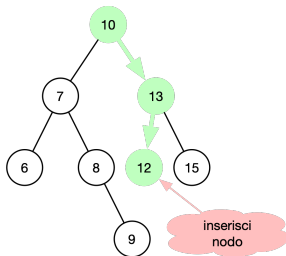
1



2



3



PSEUDOCODICE: INSERT

```
1: function INSERT(BST  $T$ , KEY  $k$ , DATA  $d$ )  $\rightarrow$  NODE
2:    $node = \text{NODE}(k, d)$ ,  $prev = \text{NIL}$ ,  $curr = T.root$ 
3:   while  $curr \neq \text{NIL}$  do                                 $\triangleright$  Search position
4:      $prev = curr$ 
5:     if  $k < curr.key$  then
6:        $curr = curr.left$ 
7:     else
8:        $curr = curr.right$ 
9:   if  $prev == \text{NIL}$  then                                     $\triangleright$  The tree is empty
10:     $T.root = node$ 
11:  else                                                         $\triangleright$  The tree is not empty
12:     $node.parent = prev$ 
13:    if  $k < prev.key$  then  $prev.left = node$ 
14:    else  $prev.right = node$ 
15:  return  $node$ 
```

■ Caso pessimo: $\Theta(h)$

■ Caso ottimo: $O(1)$

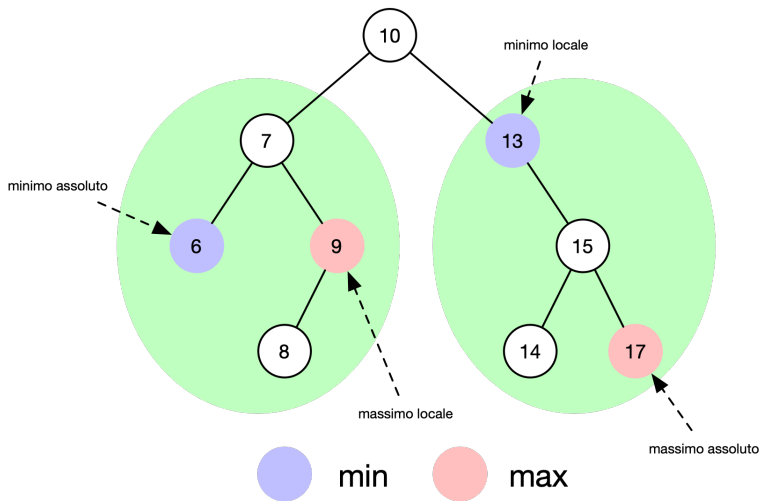
■ Ricerca posizione: $\Theta(h)$

■ Ricerca posizione: $O(1)$

■ Inserimento nodo: $O(1)$

■ Inserimento nodo: $O(1)$

ESEMPIO: MAX E MIN



PSEUDOCODICE: MAX E MIN

```
1: function MAX(NODE  $T$ )  $\rightarrow$  NODE  
2:   while  $T \neq \text{NIL}$  and  $T.\text{right} \neq \text{NIL}$  do  
3:      $T = T.\text{right}$   
4:   return  $T$ 
```

```
1: function MIN(NODE  $T$ )  $\rightarrow$  NODE  
2:   while  $T \neq \text{NIL}$  and  $T.\text{left} \neq \text{NIL}$  do  
3:      $T = T.\text{left}$   
4:   return  $T$ 
```

- Dato un sottoalbero T
 - il nodo **massimo** in T è il **nodo più a destra** in T
 - il nodo **minimo** in T è il **nodo più a sinistra** in T
- Stesso costo per entrambe le funzioni:
 - **Caso ottimo:** $O(1)$ (T non ha figlio destro (MAX) o sinistro (MIN))
 - **Caso pessimo:** $\Theta(h)$
- N.B. Non confrontiamo chiavi, usiamo solo la struttura dell'albero

ESEMPIO: PREDECESSORE (CASO 1)

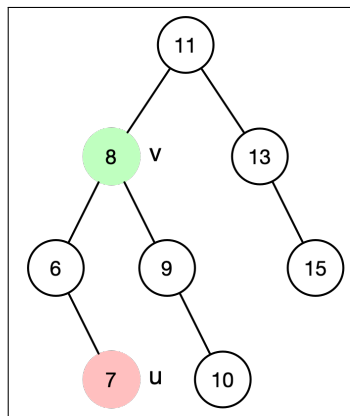
Definizione: il predecessore di un nodo v è il nodo u che precede v quando i nodi sono ordinati rispetto ad una visita in-ordine

■ Caso 1

- Il nodo v ha un figlio sinistro
- Il predecessore è il nodo u con chiave massima nel sottoalbero sinistro di v

■ Correttezza

- Per la proprietà di ordine dei BST, il sottoalbero sinistro di v contiene solo chiavi $\leq v.key$



ESEMPIO: PREDECESSORE (CASO 2)

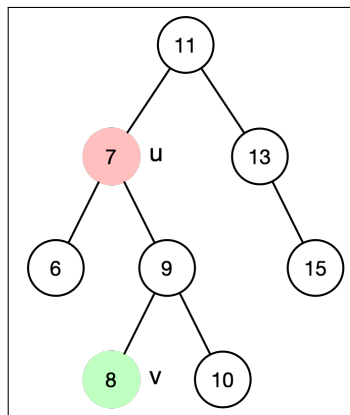
Definizione: il predecessore di un nodo v è il nodo u che precede v quando i nodi sono ordinati rispetto ad una visita in-ordine

■ Caso 2

- Il nodo v non ha un figlio sinistro
- Il predecessore è il primo antenato u tale che v stia nel sottoalbero destro di u

■ Correttezza

- Per la proprietà di ordine dei BST, il nodo v è il nodo minimo nel sottoalbero destro di u



PSEUDOCODICE: PREDECESSOR

SUCCESSOR è simmetrica a PREDECESSOR

```
1: function PREDECESSOR(Node  $T$ )  $\rightarrow$  Node
2:   if  $T == \text{NIL}$  then                                ▷ Empty tree
3:     return NIL
4:   else if  $T.\text{left} \neq \text{NIL}$  then                      ▷ Case 1
5:     return MAX( $T.\text{left}$ )
6:   else                                                  ▷ Case 2
7:      $P = T.\text{parent}$ 
8:     while  $P \neq \text{NIL}$  and  $T == P.\text{left}$  do
9:        $T = P$ 
10:     $P = P.\text{parent}$ 
11:    return  $P$ 
```

■ Caso pessimo: $\Theta(h)$

■ Caso ottimo: $O(1)$

■ Caso 1 (MAX): $\Theta(h)$

■ Caso 1 (MAX): $O(1)$

■ Caso 2: $\Theta(h)$

■ Caso 2: $O(1)$

N.B. Non confrontiamo chiavi, usiamo solo la struttura dell'albero

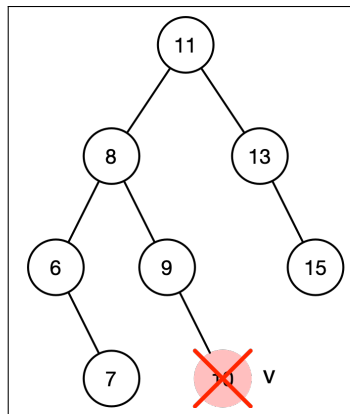
ESEMPIO: RIMOZIONE (CASO 1)

- Caso 1

- Il nodo v da rimuovere è una foglia
- Semplicemente rimuoviamo v

- Correttezza

- Se rimuoviamo una foglia non alteriamo la proprietà di ordine dei BST nei nodi rimanenti



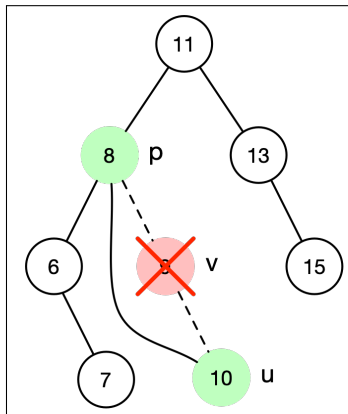
ESEMPIO: RIMOZIONE (CASO 2)

■ Caso 2

- Il nodo da rimuovere v ha un solo figlio u
- u diventa figlio del genitore p di v
- Se v è un figlio sinistro, u diventa figlio sinistro di p
- Se v è un figlio destro, u diventa figlio destro di p
- Possiamo rimuovere v

■ Correttezza

- Per la proprietà di ordine dei BST, se v è un figlio destro tutte le chiavi nel sottoalbero radicato in u sono $\geq p.key$ e se v è un figlio sinistro tutte le chiavi in u sono $\leq p.key$



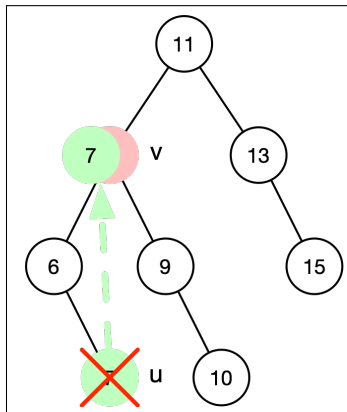
ESEMPIO: DELETE (CASO 3)

■ Caso 3

- Il nodo da rimuovere v ha due figli
- Cerchiamo il predecessore u di v
- Copiamo $v.key = u.key$ (e dati)
- Il nodo u ha **al massimo** un figlio (?)
- Rimuoviamo il nodo u (Caso 1 o 2)

■ Correttezza

- Per la proprietà di ordine dei BST, la chiave $u.key$ del predecessore di v è $\geq$ di tutte le chiavi in $v.left$ e $\leq$ di tutte le chiavi in $v.right$. Possiamo quindi sostituire v con u senza alterare la proprietà di ordine dei BST



PSEUDOCODICE: DELETE

```
1: function DELETE(BST  $T$ , KEY  $k$ )  $\rightarrow$  NODE
2:    $v$  = SEARCH( $T$ ,  $k$ )
3:   if  $v \neq \text{NIL}$  then
4:     if  $v.\text{left} \neq \text{NIL}$  and  $v.\text{right} \neq \text{NIL}$  then
5:        $u$  = PREDECESSOR( $v$ )
6:       SWAP( $v$ ,  $u$ )            $\triangleright$  Swaps  $u$  and  $v$  data and key
7:        $v$  =  $u$                 $\triangleright v$  points to the node to be removed
8:       DISCONNECT( $T$ ,  $v$ )
9:   return  $v$ 
```

- La procedura DELETE rimuove il nodo dall'albero ma non azzeri i riferimenti ai figli e al padre
 - Il riferimento al padre ci sarà utile quando vedremo Alberi AVL
- La procedura DISCONNECT gestisce solo i casi 1 e 2
 - Eventualmente aggiorna la radice dell'albero

PSEUDOCODICE: DISCONNECT

```
1: function DISCONNECT(BST T, NODE v)
2:   p = v.parent
3:   if v.right == NIL then
4:     c = v.left
5:   else
6:     c = v.right
7:   if p == NIL then                                ▷ v was the root node
8:     T.root = c
9:   else if p.left == v then                          ▷ v was a left child
10:    p.left = c
11:   else                                              ▷ v was a right child
12:    p.right = c
13:   if c ≠ NIL then c.parent = p
```

- DISCONNECT gestisce solo i casi 1 e 2
- I casi 1 e 2 richiedono un numero costante di operazioni
- *p* punta al padre di *v* mentre *c* punta al suo eventuale (unico) figlio
- Notare che il campo padre del nodo *v* non viene modificato
 - Possiamo ancora risalire fino alla radice partendo da *v*

ANALISI DI DELETE

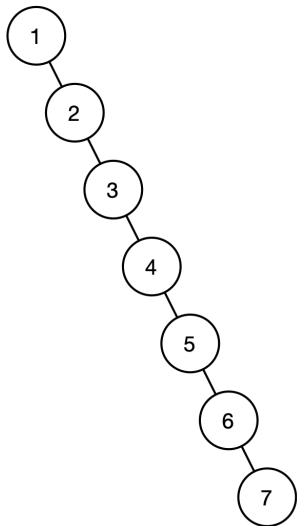
- Costo computazionale nel caso pessimo: $\Theta(h)$
 - funzione SEARCH : $\Theta(h)$
 - funzione DISCONNECT : $O(1)$
 - funzione PREDECESSOR : $\Theta(h)$
- Costo computazionale nel caso ottimo: $O(1)$
 - funzione SEARCH: $O(1)$
 - funzione DISCONNECT: $O(1)$
 - funzione PREDECESSOR: $O(1)$

ANALISI DEL CASO MEDIO

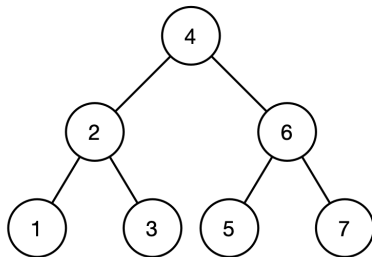
- Nel caso pessimo tutte le operazioni su BST hanno costo $\Theta(h)$
 - Il costo medio dipende dall'altezza media $\bar{h}$ di un BST
- L'altezza h di un BST può variare di molto
 - L'altezza **massima** di un BST con n nodi è $h = \Theta(n)$
 - Un BST con altezza h ha almeno $h + 1$ nodi
 - Quando l'albero binario è una lista
 - L'altezza **minima** di un BST con n nodi è $h = \Theta(\log n)$
 - Un BST con altezza h ha al massimo $2^{h+1} - 1$ nodi
 - Quando l'albero binario è *perfetto*
- Qual è l'altezza media $\bar{h}$ di un BST?
 - Caso generale (inserimenti e rimozioni)
 - Difficile da analizzare
 - Caso "*facile*": BST costruito da n inserimenti *casuali*
 - E' possibile dimostrare che $\bar{h} = O(\log n)$

ALTEZZA MASSIMA E MINIMA DI UN BST

$$n=7 \quad h=n-1=6$$



$$n=7 \quad h=\log_2(n+1)-1 = 2$$



RIPASSO: STRUTTURA DATI DIZIONARIO

- Struttura dati generica per memorizzare oggetti
 - Contiene un insieme di **chiavi** univoche
 - Ogni chiave è associata ad un **valore**
 - I valori posso essere duplicati, le chiavi sono uniche
- Operazioni basilari di un Dizionario (prototipo):
 - **SEARCH(Key k)**: cerca l'oggetto associato alla chiave k
 - **INSERT(Key k , Data d)**: aggiunge la coppia (k, d) al Dizionario
 - **DELETE(Key k)**: elimina la coppia (k, d) dal Dizionario

DIZIONARIO CON ALBERI BINARI DI RICERCA

```
1: function SEARCH(DICT D, KEY k) → DATA
2:   tmp = BSTSEARCH(D.BST, k)           ▷ search on BST
3:   if tmp == NIL then
4:     return NIL
5:   else
6:     return tmp.data
7:
8: function INSERT(DICT D, KEY k, DATA d)
9:   tmp = BSTSEARCH(D.BST, k)           ▷ search on BST
10:  if tmp == NIL then
11:    BSTINSERT(D.BST, k, d)
12:  else
13:    tmp.data = d
14:
15: function DELETE(DICT D, KEY k)
16:  BSTDELETE(D.BST, k)                 ▷ delete on BST
```

- *D.BST* è una struttura dati di tipo Albero Binario di Ricerca
- Come possiamo migliorare la procedura di inserimento?
 - Stesso costo computazionale ma più efficiente in pratica

DIZIONARIO: RIASSUNTO DEI COSTI

	SEARCH		INSERT		DELETE	
	Medio	Pessimo	Medio	Pessimo	Medio	Pessimo
Array non ordinati	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Array ordinati	$O(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	$O(n)$	$\Theta(n)$
Lista concatenata	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Albero Binario di Ricerca	$\Theta(\bar{h})$	$\Theta(h)$	$\Theta(\bar{h})$	$\Theta(h)$	$\Theta(\bar{h})$	$\Theta(h)$

- h = altezza dell'albero, $\bar{h}$ = altezza media dell'albero
 - $h = \Omega(\log n)$ e $h = O(n)$
- Non abbiamo nessun vantaggio se l'albero è sbilanciato e $h = \Theta(n)$
- Tutte le operazioni hanno un costo logaritmico se $h = \Theta(\log n)$
- **Idea:** mantenere l'altezza dell'albero logaritmica sul numero di nodi